

Credit Card Approval Prediction Using Machine Learning

Project Description:

Credit Card Approval Prediction Using Machine Learning is an intelligent web-based application that leverages Machine Learning to automate the credit card approval process by predicting whether an applicant is eligible for a credit card based on financial and personal information. The platform provides a user-friendly interface where users can enter applicant details and instantly receive an approval or rejection prediction. In addition to the prediction, the system offers a detailed explanation highlighting the key factors that influenced the decision, enabling users to better understand the outcome. The application also features an interactive dashboard that presents project information, machine learning model details, system workflow, and performance evaluation metrics, providing a comprehensive overview of the prediction process.

Built using the **Decision Tree** machine learning algorithm, the application is developed with **Python, Flask, HTML, CSS, and JavaScript**, ensuring accurate predictions and a responsive user experience. The trained model is integrated into the Flask backend using **Joblib**, allowing fast and efficient real-time predictions without retraining the model. The application incorporates input validation, secure data processing, and a modular architecture to enhance reliability and maintainability. Deployed as a web application, the system demonstrates how machine learning can assist financial institutions in making faster, more consistent, and data-driven credit approval decisions while reducing manual effort and improving operational efficiency.

Scenarios

Scenario 1:

A bank officer receives a credit card application from a salaried employee with a stable annual income, good credit history, and low existing debt. The officer enters the applicant's details into the Credit Card Approval Prediction system. The application analyzes the input using the trained Decision Tree model and instantly predicts that the application is **Approved**. The system also displays the key reasons for approval, such as a strong credit history, sufficient income, and a favorable debt-to-income ratio, enabling the officer to make a quick and informed decision.

Scenario 2:

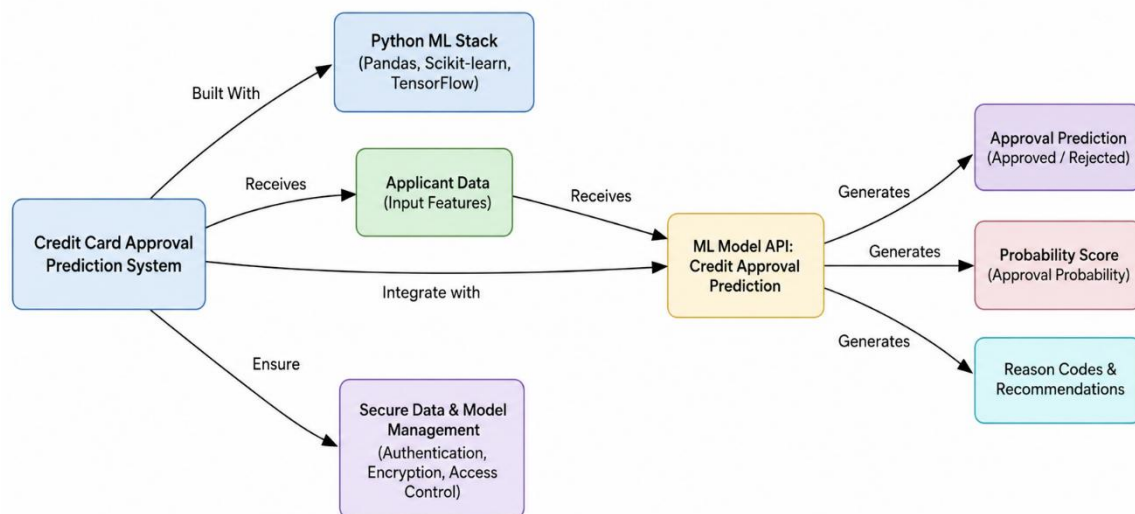
A customer applies for a credit card but has a low annual income, multiple outstanding loans, and a poor credit history. After entering the applicant's information into the system, the machine learning model predicts that the application is **Rejected**. Along with the prediction, the application provides

a detailed explanation identifying the major factors influencing the rejection, such as low income, high debt burden, and unfavorable credit history, helping the user understand the decision.

Scenario 3:

A financial institution processes multiple credit card applications daily and requires a faster, more consistent evaluation process. Using the Credit Card Approval Prediction system, loan officers enter applicant details through the web interface and receive real-time approval or rejection predictions along with decision explanations. This enables the institution to reduce manual effort, improve decision consistency, and accelerate the credit card approval process while supporting data-driven decision-making.

Technical Architecture:



Description: The Credit Card Approval Prediction System follows a modular machine learning architecture that integrates a user-friendly web interface with a predictive ML model to automate the credit card approval process. The application is developed using the Flask web framework, where users submit applicant details through an input form. These input features are securely processed and passed to a trained Decision Tree machine learning model for prediction. The backend ensures secure data and model management while handling feature preprocessing and model inference. Based on the applicant's information, the system generates an approval decision (Approved/Rejected), an approval probability score, and detailed reason codes with personalized recommendations, providing transparency and interpretability in the prediction process.

Pre-requisites:

- **Flask Framework Knowledge:** Understanding of Flask for developing the web application, handling routes, templates, and backend integration.
- **Machine Learning Fundamentals:** Knowledge of supervised learning, classification algorithms, model training, evaluation, and prediction using Scikit-learn.
- **Python Programming Proficiency:** Familiarity with Python programming, including data structures, functions, file handling, and object-oriented programming.
- **Data Analysis Libraries:** Experience with **Pandas** and **NumPy** for data preprocessing, feature engineering, and dataset manipulation.
- **Machine Learning Library (Scikit-learn):** Knowledge of Scikit-learn for data preprocessing, model training, testing, serialization (Joblib), and prediction.
- **HTML, CSS, and JavaScript Skills:** Understanding of front-end technologies to design responsive user interfaces and interactive web pages.
- **Version Control with Git and GitHub:** Familiarity with Git commands and GitHub for source code management, collaboration, and project hosting.
- **Development Environment Setup:** Installation and configuration of Python, Flask, required libraries, and a virtual environment for project development.
- **Model Deployment:** Basic knowledge of deploying Flask applications using platforms such as **Render**, along with managing environment variables and project dependencies.

Project Workflow

Milestone 1: Data Preparation and Model Development

- **Activity 1.1:** Collect the credit card application dataset and perform data preprocessing, including handling missing values, encoding categorical variables, feature scaling (if required), and splitting the dataset into training and testing sets.
- **Activity 1.2:** Research and evaluate suitable machine learning classification algorithms for credit card approval prediction. Select the **Decision Tree Classifier** based on its performance and interpretability.
- **Activity 1.3:** Train the Decision Tree model, evaluate its performance using appropriate metrics (Accuracy, Precision, Recall, F1-Score), and save the trained model using Joblib for deployment.
- **Activity 1.4:** Set up the development environment by installing Python, Flask, Scikit-learn, Pandas, NumPy, Joblib, and other required libraries.

Milestone 2: Backend Development

- **Activity 2.1:** Develop the Flask backend to load the trained Decision Tree model and process user input received from the web application.
- **Activity 2.2:** Implement data preprocessing within the backend to transform user inputs into the format expected by the machine learning model.

- **Activity 2.3:** Generate prediction results, including the approval status (Approved/Rejected), approval probability score, and decision explanations with personalized recommendations.

Milestone 3: Application Logic (app.py)

- **Activity 3.1:** Develop the main application logic in **app.py**, configuring application routes for the dashboard, prediction form, and result page.
- **Activity 3.2:** Integrate the trained Decision Tree model with Flask to perform real-time predictions based on applicant information.
- **Activity 3.3:** Implement error handling and input validation to ensure reliable and secure application performance.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop a responsive user interface using HTML, CSS, and JavaScript with a modern dashboard layout.
- **Activity 4.2:** Create an interactive application form for collecting applicant information and validating user inputs.
- **Activity 4.3:** Develop a comprehensive result page displaying:
 - Approval Status (Approved/Rejected)
 - Approval Probability Score
 - Prediction Confidence
 - Key Factors Influencing the Decision
 - Personalized Recommendations
 - Decision Tree Visualization (where applicable)
- **Activity 4.4:** Enhance user experience with responsive design, animations, informative cards, and visual indicators.

Milestone 5: Testing and Deployment

- **Activity 5.1:** Test the application using multiple applicant profiles to verify prediction accuracy, input validation, and overall system performance.
- **Activity 5.2:** Prepare the application for deployment by organizing project files, managing dependencies through **requirements.txt**, and configuring environment settings.
- **Activity 5.3:** Deploy the Flask application on **Render** to provide secure public access and verify end-to-end functionality after deployment.

Milestone 6: Documentation and Conclusion

- **Activity 6.1:** Prepare comprehensive project documentation, including system architecture, workflow, implementation details, model evaluation, and deployment process.
- **Activity 6.2:** Summarize the project outcomes, discuss system advantages, limitations, and propose future enhancements such as integrating ensemble models, explainable AI (XAI) techniques, and real-time banking APIs.

Milestone 1: Model Selection and Architecture

In this milestone, the primary focus is on selecting an appropriate machine learning algorithm for predicting credit card approval and designing the overall architecture of the application. This involves understanding the project requirements, analyzing the available dataset, comparing different classification algorithms, and defining the interaction between the frontend, backend, and machine learning model. The objective is to build a reliable, accurate, and user-friendly credit card approval prediction system.

Activity 1.1: Dataset Collection and Analysis

Before developing the prediction model, an appropriate dataset containing applicant information and approval status was selected and analyzed. The following steps were performed:

- **Step 1 — Collect the Dataset:** Obtain a publicly available credit card approval dataset containing applicant financial and personal information.
- **Step 2 — Analyze Dataset Features:** Examine all input attributes such as annual income, employment status, education level, credit history, loan amount, debt ratio, and other relevant financial indicators.
- **Step 3 — Handle Missing Values:** Clean the dataset by removing duplicate records, handling missing values, and correcting inconsistent data entries.
- **Step 4 — Encode Categorical Variables:** Convert categorical features into numerical values using appropriate encoding techniques to make them suitable for machine learning algorithms.
- **Step 5 — Split the Dataset:** Divide the processed dataset into training and testing sets to evaluate model performance effectively.
- **Step 6 — Verify Data Quality:** Ensure the dataset is balanced, complete, and ready for machine learning model training.

Activity 1.2: Research and Select the Appropriate Machine Learning Model

The following activities were carried out to identify the most suitable classification algorithm for the project:

- **Understand the Project Requirements:** Analyze the need for an automated credit card approval system capable of making accurate and explainable predictions.
- **Explore Classification Algorithms:** Study commonly used classification algorithms such as Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and Support Vector Machine (SVM).
- **Evaluate Model Performance:** Compare algorithms using evaluation metrics such as Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.
- **Select the Optimal Model:** Choose the **Decision Tree Classifier** due to its high prediction accuracy, simple implementation, fast prediction speed, and ability to provide understandable decision rules and explanations for approval or rejection.

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Design the overall system architecture illustrating the interaction between the user interface, Flask backend, machine learning model, and prediction results.
- **Detail Frontend Functionality:** Define how users enter applicant information through the web interface and submit prediction requests.
- **Outline Backend Responsibilities:** Specify how the Flask backend validates user input, preprocesses applicant data, loads the trained Decision Tree model using Joblib, performs predictions, and generates approval or rejection reasons.
- **Describe Machine Learning Integration:** Explain how the backend processes user input, converts it into the required feature format, passes it to the trained model, interprets the prediction result, and returns both the prediction and decision explanation to the frontend.

Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python **3.10 or above** is installed together with the latest version of pip.
- **Create a Virtual Environment:** Create and activate a virtual environment to isolate project dependencies.

D:\Projects> python -m venv myenv

D:\Projects> myenv\Scripts\activate

(myenv) D:\Projects> pip install -r requirements.txt

- **Install Flask:**

(myenv) D:\Projects> pip install Flask

- **Install Machine Learning Libraries:**

(myenv) D:\Projects> pip install scikit-learn pandas numpy joblib

- **Install Additional Dependencies:**

(myenv) D:\Projects> pip install matplotlib seaborn

(Install only the libraries your project actually uses.)

- **Verify Project Structure:** Ensure the project directory contains all required files and folders, including:

Credit Card Approval Prediction/

```
|
|— app.py
|— requirements.txt
|— decision_tree_model.pkl
|— dataset.csv
|— templates/
|— static/
|   |— css/
|   |— js/
```

└─ images/
└─ README.md

- **Run the Application:** Start the Flask development server and verify that the application launches successfully before proceeding with further development.

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Prediction Features

Implement the core functionalities required for predicting credit card approval and providing users with meaningful decision insights through a single machine learning prediction process.

- **Credit Card Approval Prediction:** Develop the primary prediction module that accepts applicant details, preprocesses the input data, and predicts whether the credit card application should be **Approved** or **Rejected** using the trained Decision Tree machine learning model.
- **Decision Explanation Module:** Implement a feature that analyzes the prediction result and displays the major factors contributing to the approval or rejection decision, helping users understand the reasoning behind the model's output.
- **Input Validation Module:** Develop input validation to ensure that all mandatory applicant details are entered correctly before prediction. Invalid, missing, or incorrectly formatted values are rejected with appropriate error messages to improve system reliability.

Activity 2.2: Implement the Flask Backend

Define Routes in Flask

- Set up routes in **app.py** for the home page, dashboard page, prediction form, and prediction result page.

Process User Input

- Create a prediction form in **index.html** where users can enter applicant information such as age, annual income, employment status, credit history, loan amount, debt ratio, and other required attributes.
- Use **Flask's request.form** (or `request.get_json()` if using AJAX) to retrieve the applicant information submitted from the frontend.
- Validate all user inputs to ensure mandatory fields are completed, numerical values are within valid ranges, and incorrect or incomplete submissions are rejected before prediction.
- Convert categorical input values into numerical representations required by the trained machine learning model.

Integrate the Machine Learning Model

- Load the trained **Decision Tree** model using the **Joblib** library when the Flask application starts.
- Preprocess the applicant information into the same feature format used during model training.
- Pass the processed feature vector to the trained Decision Tree model using the **predict()** method.
- Generate the prediction result (**Approved** or **Rejected**) and determine the corresponding explanation based on the applicant's financial profile and model output.
- Return the prediction result and decision explanation to the frontend, where they are displayed on the result page in a user-friendly format.

Display Prediction Results

- Show the prediction status using clear visual indicators such as **Approved** or **Rejected**.
- Display detailed reasons explaining the prediction outcome, highlighting factors such as annual income, credit history, existing debt, employment status, or other significant applicant attributes.

- Present the prediction results in an organized and responsive interface, allowing users to easily interpret the model's decision.

Milestone 3: App.py Development

Milestone 3 focuses on developing and refining the core logic of the **app.py** file, which serves as the backbone of the Credit Card Approval Prediction application. In this milestone, Flask routes are implemented to manage navigation and prediction requests, the trained Decision Tree model is integrated into the backend, and applicant data is processed to generate real-time approval predictions. Unit testing is also performed to verify that each feature functions correctly and reliably.

Activity 3.1: Writing the Main Application Logic in app.py

Define the Core Routes in app.py

Establish routes for the application's main pages and prediction functionality.

Home Page Route

The home route renders the main dashboard of the application.

```
@app.route("/")
def home():
    return render_template("index.html")
```

Prediction Route

Create a POST route that accepts applicant information from the prediction form, validates the input, loads the trained Decision Tree model, performs prediction, and returns the result.

```
@app.route("/predict", methods=["POST"])
def predict():
    try:
        # Read form data
        applicant_data = request.form

        # Validate input
        if not applicant_data:
            return render_template(
                "result.html",
```

```
        prediction="Invalid Input"
    )

    # Preprocess input data
    processed_data = preprocess_input(applicant_data)

    # Load trained model
    prediction = model.predict([processed_data])

    # Generate explanation
    reasons = generate_reason(applicant_data, prediction)

    return render_template(
        "result.html",
        prediction=prediction[0],
        reasons=reasons
    )

except Exception as e:
    return render_template(
        "result.html",
        prediction="Error",
        message=str(e)
    )
```

Set Up Route Handlers

Implement the following responsibilities for the prediction route:

- Accept applicant information submitted through the HTML prediction form.
- Retrieve user input using **Flask's request.form**.
- Validate that all mandatory fields are completed.
- Verify that numerical values such as income, age, loan amount, and debt ratio are within acceptable ranges.
- Reject invalid submissions and display meaningful error messages.
- Convert categorical values into numerical format required by the machine learning model.

- Pass the processed feature vector to the trained Decision Tree classifier.
- Display the prediction result together with the explanation for approval or rejection.

Define Input Processing Functions

Input Preprocessing

Develop a preprocessing function that converts raw applicant information into the numerical feature vector expected by the trained model.

```
def preprocess_input(form_data):  
    """  
    Convert user input into the format required by  
    the trained Decision Tree model.  
    """  
  
    processed_features = [  
        ...  
    ]  
  
    return processed_features
```

The preprocessing function performs the following tasks:

- Reads applicant information from the submitted form.
- Encodes categorical variables.
- Converts numerical values into appropriate data types.
- Arranges features in the same order used during model training.

Integrate the Machine Learning Model

Load the Trained Model

Load the trained Decision Tree model using the Joblib library when the application starts.

```
import joblib  
  
model = joblib.load("decision_tree_model.pkl")
```

Generate Predictions

Use the loaded model to predict the credit card approval status.

```
prediction = model.predict([processed_data])
```

The prediction returns one of the following values:

- **Approved**
- **Rejected**

Generate Decision Explanation

Develop a helper function that generates meaningful reasons for the prediction result based on the applicant's information.

```
def generate_reason(data, prediction):
```

```
    reasons = []
```

```
    if prediction == "Approved":
```

```
        reasons.append("Good credit history")
```

```
        reasons.append("Stable annual income")
```

```
        reasons.append("Low debt ratio")
```

```
    else:
```

```
        reasons.append("Poor credit history")
```

```
        reasons.append("High outstanding debt")
```

```
        reasons.append("Insufficient annual income")
```

```
    return reasons
```

This function:

- Identifies the major factors influencing the prediction.
- Generates understandable explanations.
- Improves transparency of the machine learning decision.

Return Prediction Results

After prediction is completed, the backend returns the following information to the frontend:

- Approval Status (**Approved** or **Rejected**)
- Decision Explanation
- Applicant Summary (if applicable)

The frontend displays these results on the **Result Page** using a clear and user-friendly layout.

Error Handling

Implement exception handling throughout the application to improve reliability.

Examples include:

- Missing applicant information
- Invalid numerical values
- Model loading failures
- Prediction errors
- Unexpected runtime exceptions

Meaningful error messages are displayed instead of application crashes.

Unit Testing

Test the application's core functionalities to verify correct operation.

The following scenarios are tested:

- Valid applicant information produces a prediction successfully.
- Invalid or incomplete input is rejected.
- Prediction results are displayed correctly.

- Approval reasons and rejection reasons are generated accurately.
- Application handles unexpected errors gracefully.

Application Routes

Route	Purpose
/	Displays the Dashboard/Home page
/predict (POST)	Accepts applicant information, performs prediction using the Decision Tree model, and displays the result with decision explanations.

Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly, responsive, and visually appealing web interface for the Credit Card Approval Prediction System. This involves designing an interactive dashboard, a well-structured prediction form, and a detailed result page using HTML, CSS, and JavaScript. The frontend ensures a smooth user experience while allowing users to submit applicant information and instantly view the prediction results with decision explanations.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure

- Develop the main HTML pages including the Dashboard, Prediction Form, and Result Page using semantic HTML elements.
- Design a responsive navigation bar containing the project title and navigation links for easy access to different sections of the application.
- Create a hero section on the dashboard introducing the Credit Card Approval Prediction System along with its objectives and key features.
- Organize the application into separate sections including:
 - Dashboard
 - About the Project
 - Machine Learning Model Information
 - Prediction Form
 - Prediction Result
 - Team Information
 - Contact Section
- Ensure proper use of semantic HTML tags to improve readability, accessibility, and maintainability.

Design a Responsive Layout Using CSS

- Create a CSS stylesheet (static/css/style.css) implementing a modern and responsive design with an attractive color scheme and smooth animations.
- Use Flexbox and CSS Grid layouts to organize dashboard sections, feature cards, prediction forms, and result panels.
- Design responsive cards for displaying:
 - Project Features
 - Machine Learning Model Details
 - Prediction Result
 - Approval/Rejection Reasons
- Add hover effects, transitions, and animations to improve the overall user experience.
- Implement media queries to ensure the application adapts seamlessly across desktop, tablet, and mobile devices.

Create Separate UI Components

Develop dedicated user interface components for each major feature of the application.

Dashboard Components

- Interactive hero section with project overview.
- Feature cards describing system capabilities.
- Machine learning model information panel.
- Navigation menu for quick access to different sections.

Prediction Form

- Responsive input form for entering applicant details.
- Appropriate input controls including text boxes, number fields, and dropdown menus.
- Client-side validation to reduce invalid submissions.

Result Page

- Prediction status displayed prominently as Approved or Rejected.
- Color-coded result card for easy interpretation.
- Detailed explanation showing the primary reasons influencing the prediction.
- Button to return to the prediction form for another evaluation.

Activity 4.2: Creating Dynamic User Interactions with JavaScript

Handle Form Submission

- Implement JavaScript functions to validate applicant information before submission.
- Display appropriate warning messages if mandatory fields are missing or contain invalid values.
- Prevent duplicate form submissions by temporarily disabling the Predict button while processing.

Display Prediction Results

- Dynamically display the prediction outcome after receiving the response from the Flask backend.

- Update the result page with:
 - Prediction Status
 - Decision Explanation
 - Applicant Summary (if applicable)
- Highlight approval and rejection results using different visual styles to improve readability.

Improve User Experience

- Display loading indicators while the prediction request is being processed.
- Automatically scroll to the prediction result section after successful prediction.
- Allow users to easily perform another prediction without refreshing the application.
- Ensure all interactive elements respond smoothly through JavaScript event handling.

Restore Interface State

- Re-enable the Predict button after processing is complete.
- Clear validation messages once the user corrects invalid inputs.
- Maintain a responsive and consistent interface regardless of whether the prediction succeeds or fails.

Frontend Technologies Used

- HTML5 – Structure of the web pages.
- CSS3 – Responsive layout, styling, animations, and visual effects.
- JavaScript (Vanilla JS) – Client-side validation, dynamic interactions, and improved user experience.
- Bootstrap (if used) – Responsive components and grid layout.
- Font Awesome – Icons used throughout the application.

Milestone 5: Deployment

In Milestone 5, the focus is on deploying the **Credit Card Approval Prediction** application for both local execution and cloud hosting. The application is first tested locally to ensure all functionalities work correctly, and then deployed on **Render** to provide public access through a secure web URL. This milestone also includes configuring dependencies, verifying deployment, and validating the application's performance in the hosted environment.

Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment

- Open **Command Prompt** or **Windows Terminal**.
- Navigate to the project directory.
- Create and activate a virtual environment.

```
D:\Projects> python -m venv myenv
```

```
D:\Projects> myenv\Scripts\activate
(myenv) D:\Projects> pip install -r requirements.txt
```

Install Required Dependencies

Install all required Python libraries listed in **requirements.txt**.

```
(myenv) D:\Projects> pip install -r requirements.txt
```

The application installs dependencies such as:

- Flask
- Pandas
- NumPy
- Scikit-learn
- Joblib

Verify Project Structure

Ensure the project directory contains the required files and folders.

Credit Card Approval Prediction

```
├── app.py
├── requirements.txt
├── card_model.pkl
├── dataset.csv
├── templates/
├── static/
├── README.md
└── .gitignore
```

Activity 5.2: Local Testing and Verification

Start the Flask Development Server

Run the application locally.

```
(myenv) D:\Projects> python app.py
```

Access the Application

Open a web browser and visit: **http://127.0.0.1:5000**

Verify Application Features

Test the following functionalities:

- Open the dashboard successfully.
- Navigate to the prediction form.
- Enter valid applicant information.
- Generate an approval or rejection prediction.
- Verify that decision reasons are displayed correctly.
- Test invalid inputs to ensure validation messages appear.
- Confirm the application responds without errors.

Activity 5.3: Public Deployment on Render

The application is deployed on **Render**, allowing users to access it through a publicly available web URL without installing any software.

Prepare the Project Repository

- Push the complete project to a GitHub repository.
- ```
git add .
git commit -m "Final deployment version"
git push origin main
```

#### Deploy Using Render

- Create an account on **Render**.
- Connect the GitHub repository.
- Create a **New Web Service**.
- Select the Credit Card Approval Prediction repository.

Configure the deployment settings:

##### Build Command :

```
pip install -r requirements.txt
```

##### Start Command :

```
python app.py
```

#### Access the Hosted Application

After successful deployment, Render generates a public URL similar to:

<https://credit-card-approval-prediction-3cw7.onrender.com>

Users can access the application directly through this URL using any modern web browser.

#### Deployment Verification

After deployment, verify the following:

- Dashboard loads successfully.
- Prediction form accepts applicant details.
- Machine learning model generates predictions correctly.
- Approval or rejection reasons are displayed.
- All CSS, JavaScript, and images load correctly.
- No runtime errors occur during prediction.

#### Important Notes

- Ensure all required Python packages are listed in **requirements.txt** before deployment.
- Keep the trained Decision Tree model (card\_model.pkl) inside the project directory so Render can load it during execution.
- Push all code changes to GitHub before triggering a new deployment.
- Free Render services may enter a sleep state after periods of inactivity, causing the first request to take slightly longer while the service wakes up.

- If the application is updated, push the latest changes to GitHub and redeploy the project on Render to apply the modifications.

## Exploring the Website's Web Pages:

### Dashboard (Home Page)



#### Description:

The **Dashboard** serves as the landing page of the Credit Card Approval Prediction System and provides users with an attractive introduction to the application. It features a modern, responsive interface with a visually appealing background, navigation bar, and well-organized content sections. The dashboard presents an overview of the project, its objectives, key features, technology stack, machine learning model information, and project workflow. Users can easily navigate to the prediction form through the prominent "**Predict Now**" button, while additional sections provide details about the development team and contact information. The clean layout and responsive design ensure an intuitive user experience across desktop and mobile devices.

## Prediction Form Page

 **CreditAI** [← Back to Dashboard](#)

 **Credit Card Approval Prediction**  
Fill in the applicant's details below to estimate the credit card approval decision.

 **Applicant Information**

Gender  

Select Gender

Age (Years)  

Example : 30

Number of Children  

0

Family Members  

2

Family Status  

Select

 **Financial Information**

Annual Income  

Example : 250000

Income Type  

Select

Years Employed  

Example : 5

 **Education & Occupation**

Education  

Select

Occupation  

Select

 **Property Information**

Own Car  

Select

Own House / Property  

Select

Housing Type  

Select

 **Contact Information**

Mobile Available  

Yes

Work Phone  

Yes

Personal Phone  

Yes

Email Available  

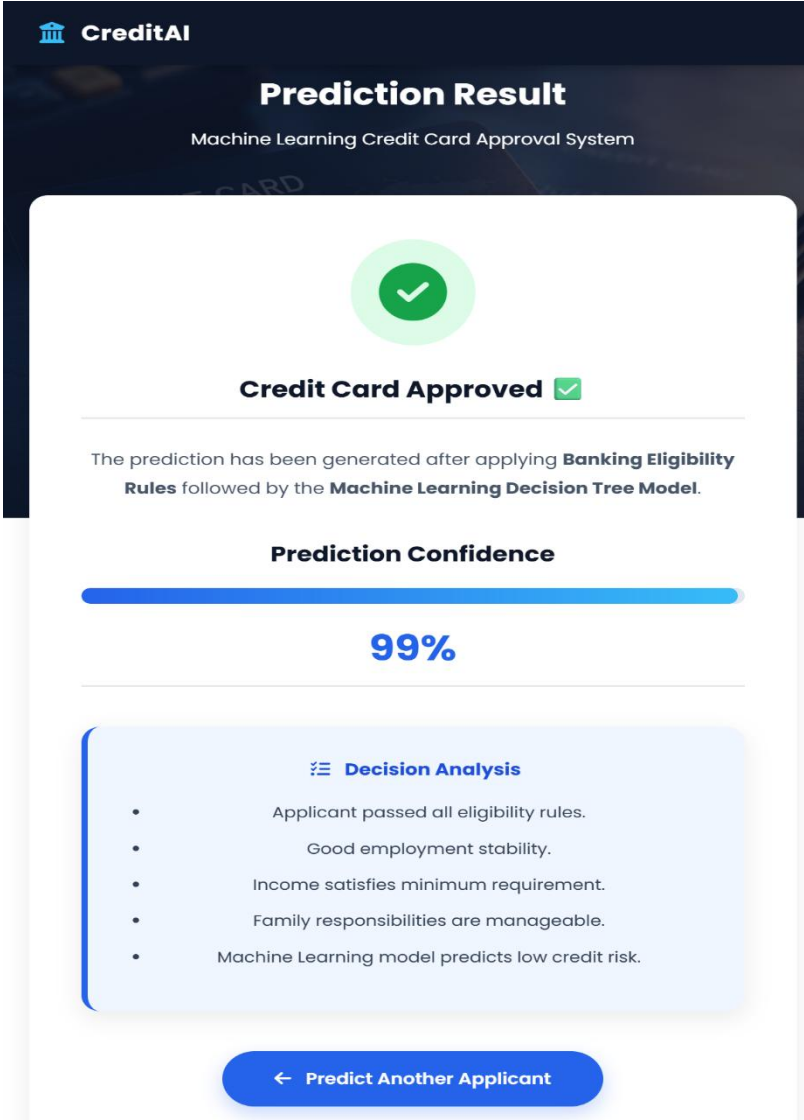
Yes

 **Predict Approval**

### Description:

The **Prediction Form** is the primary input interface of the application where users enter applicant information required for credit card approval prediction. The form is designed using responsive cards and organized input fields, allowing users to provide details such as annual income, employment status, education level, credit history, loan amount, debt ratio, and other applicant attributes. Client-side validation ensures that all mandatory fields are completed with valid values before submission. After reviewing the entered information, users can click the "**Predict**" button to securely submit the application for machine learning analysis.

### Prediction Result Page



The screenshot displays the 'Prediction Result' page of the CreditAI application. At the top, the CreditAI logo is visible. The main heading is 'Prediction Result', with the subtitle 'Machine Learning Credit Card Approval System'. A large green checkmark icon indicates a successful prediction. Below this, the text 'Credit Card Approved' is followed by a small green checkmark. A message states: 'The prediction has been generated after applying **Banking Eligibility Rules** followed by the **Machine Learning Decision Tree Model**.' A 'Prediction Confidence' section features a blue progress bar and the value '99%'. A 'Decision Analysis' section, marked with a list icon, contains five bullet points: 'Applicant passed all eligibility rules.', 'Good employment stability.', 'Income satisfies minimum requirement.', 'Family responsibilities are manageable.', and 'Machine Learning model predicts low credit risk.' At the bottom, a blue button with a left arrow and the text 'Predict Another Applicant' is shown.

**CreditAI**

## Prediction Result

Machine Learning Credit Card Approval System

**Credit Card Approved** ✓

The prediction has been generated after applying **Banking Eligibility Rules** followed by the **Machine Learning Decision Tree Model**.

**Prediction Confidence**

99%

**Decision Analysis**

- Applicant passed all eligibility rules.
- Good employment stability.
- Income satisfies minimum requirement.
- Family responsibilities are manageable.
- Machine Learning model predicts low credit risk.

← Predict Another Applicant

**Description:**

After processing the applicant information, the **Prediction Result Page** displays the machine learning prediction in a clear and user-friendly format. The page prominently indicates whether the credit card application is **Approved** or **Rejected** using color-coded result panels for easy interpretation. In addition to the prediction outcome, the application provides a detailed explanation highlighting the major factors that influenced the decision, enabling users to understand the reasoning behind the model's prediction. The result page also includes options to return to the prediction form and perform additional predictions without restarting the application.

**Conclusion**

The **Credit Card Approval Prediction Using Machine Learning** system is an intelligent web application developed using **Python, Flask, HTML, CSS, JavaScript**, and the **Decision Tree** machine learning algorithm to automate the credit card approval process. The application enables users to enter applicant information through a responsive web interface and instantly receive an approval or rejection prediction accompanied by detailed decision explanations. The project demonstrates how machine learning can support faster, more consistent, and data-driven credit approval decisions while improving transparency through explainable prediction results.

The application includes an interactive dashboard, a user-friendly prediction form, comprehensive result visualization, and robust input validation to ensure reliability and ease of use. Deployed as a web application and supported by a modular architecture, the system provides a strong foundation for future enhancements. Potential improvements include integrating larger real-world datasets, implementing secure user authentication, adding a database for storing prediction history, incorporating advanced machine learning algorithms, and connecting with banking APIs for real-time financial decision support. With these enhancements, the system can evolve from an academic prototype into a practical decision-support solution for financial institutions.